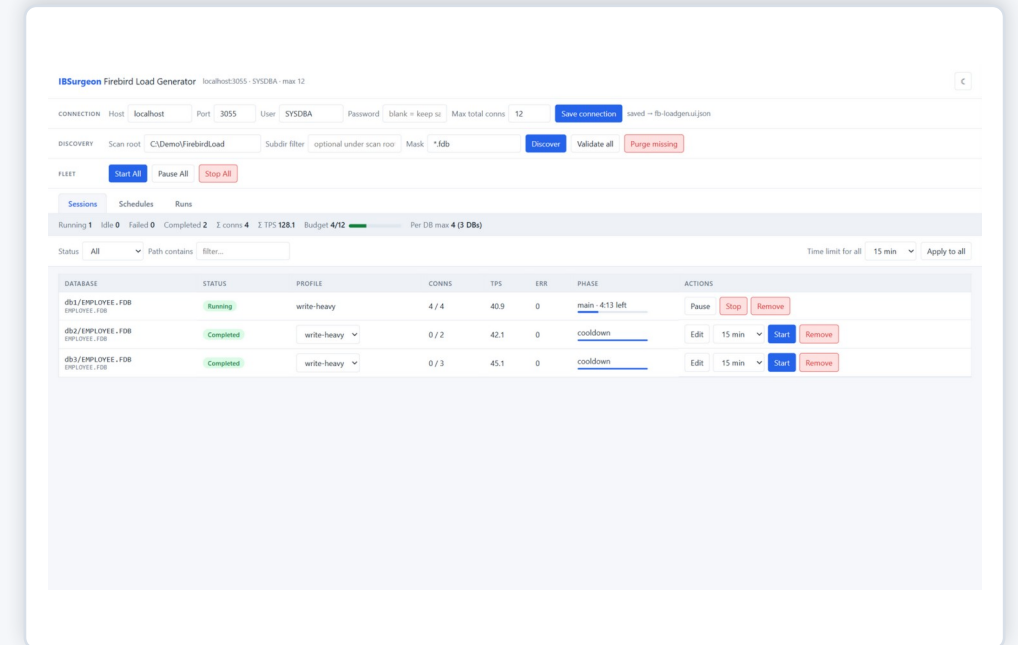


IBSurgeon Firebird Load Generator

Load testing for Firebird — from a one-shot CLI to a scheduled, multi-database fleet with a full REST API.

EMPLOYEE-schema workloads · ramp + spike profiles · schedules · run history · Prometheus & webhooks



Multi-database control plane — live status, one click per database

Executive summary

1

Go binary — CLI, UI and API in one executable

4

workload profiles: write-heavy, read-heavy, spike, oltp-emul

10+

databases per control plane, one worker = one connection

100%

API-first: schedules, runs and metrics without the UI

TWO WAYS TO RUN

One-shot CLI

```
fb-loadgen --profile write-heavy \  
  --dsn "localhost/3055:./EMPLOYEE.FDB" \  
  --warmup 30 --main 120 --cooldown 20
```

Ramp, measure, report — one database, one run

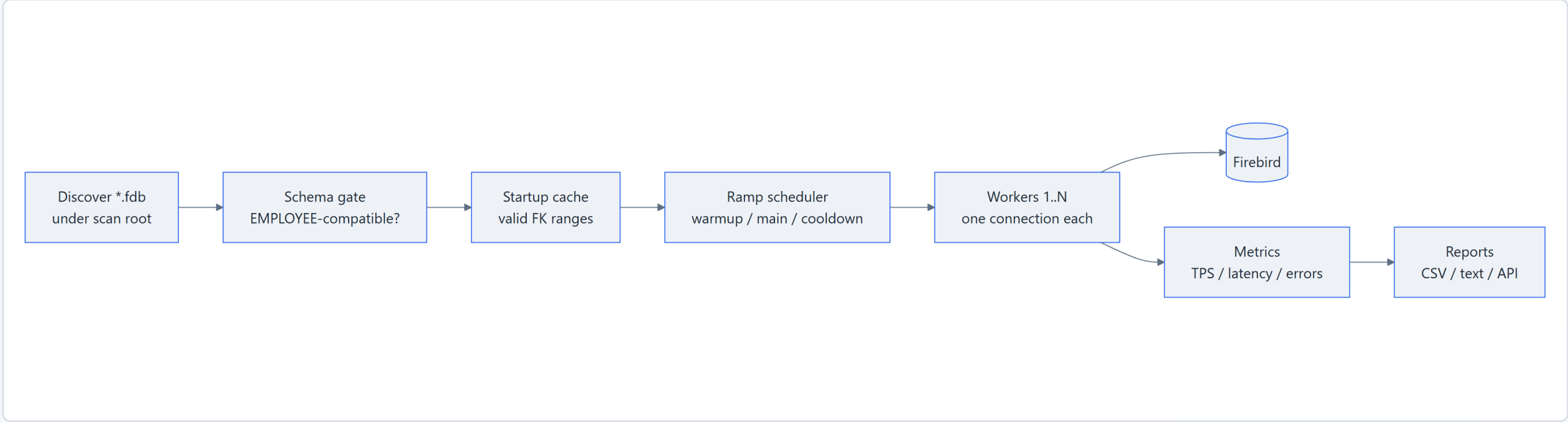
Control plane

```
fb-loadgen --ui    # UI + REST API + schedules  
curl $BASE/api/sessions/$ID/start \  
  -d '{"timeLimitMin": 15}'
```

Discover many databases, run and schedule a fleet

How it works — the life of one run

One process, one worker = one dedicated connection. Gate and cache run at every start.

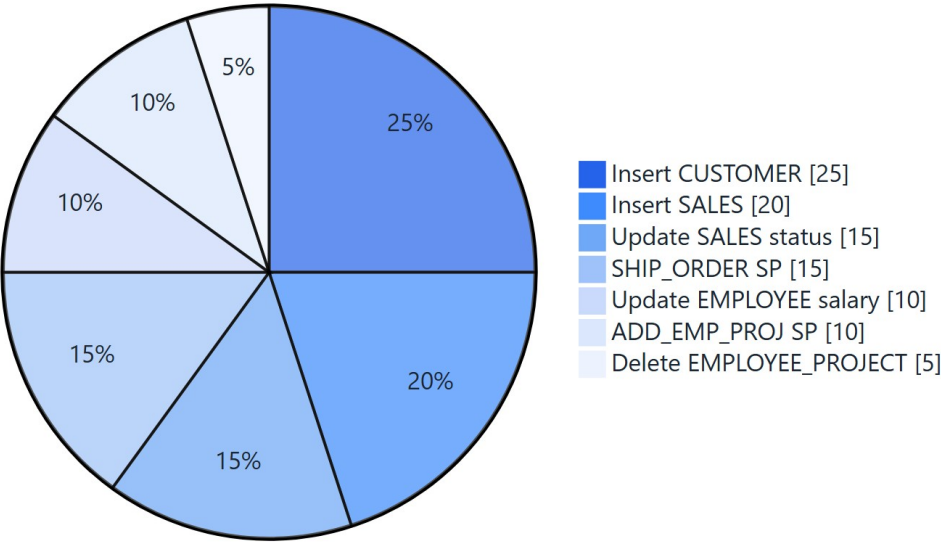


Everything runs inside one Go binary — the same executable is the CLI, the control plane and the load generator.

Workload model — real operation weights

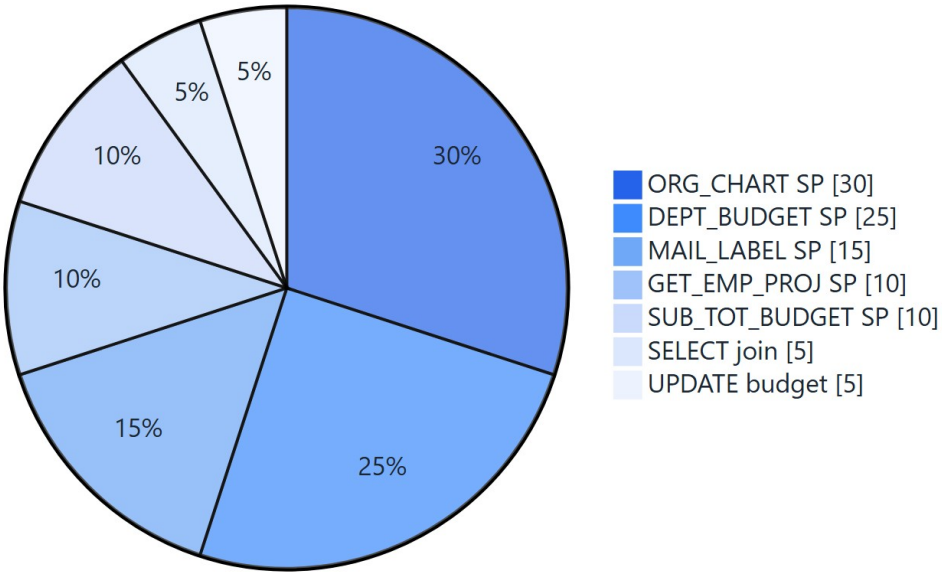
Schema-aware operations: inserts respect FKs, updates follow state transitions, expected business exceptions are not errors.

Write-heavy (weights from profile definition)



Respects EMPLOYEE constraints (PO_NUMBER, salary bounds, status transitions)

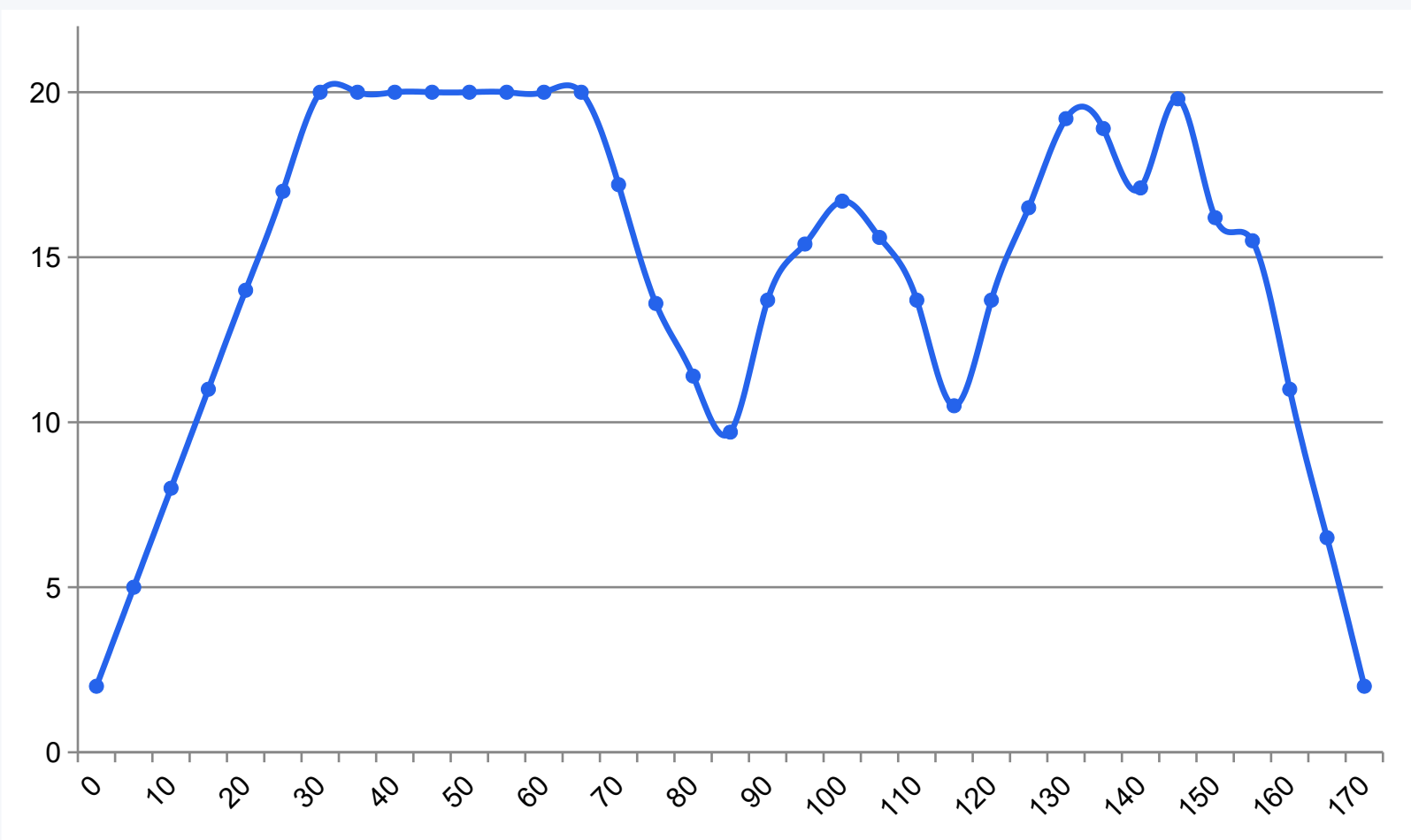
Read-heavy (weights from profile definition)



Business exceptions like order_already_shipped tracked separately

Connection ramp

Warmup (linear up) → main (random walk between min and max) → cooldown (linear down). A time limit rescales all phases.



no-limit mode: 60 s warmup, then steady until stopped

Warmup 0 → 30 s
linear ramp to the peak

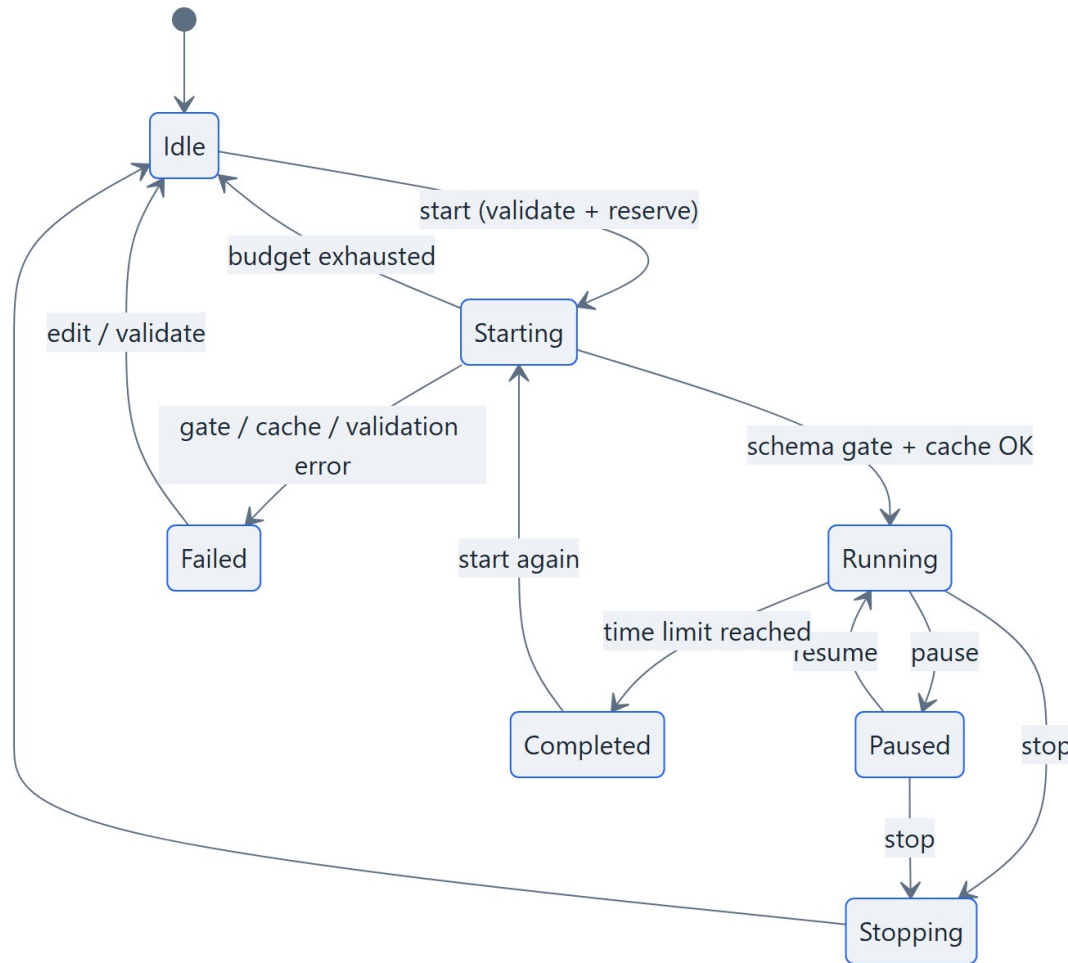
Main 30 → 150 s
random walk between min/max

Cooldown 150 → 170 s
graceful ramp down

spike profile = sawtooth cycles

Session lifecycle

Every database in the fleet is a session with an explicit, observable state.



Starting — schema gate + connection-budget reservation

Running — load is being generated; live TPS and latency

Paused — workers and countdown frozen, connections kept

Completed — time limit reached — final metrics retained

Failed — connection or schema problem, reason in lastError

Safety rails & compatibility

Schema gate — a database must be reachable and EMPLOYEE-readable before any load

Startup lookup cache — preloads valid FK ranges — inserts never guess

Fleet connection budget — maxTotalConns, split evenly, resized live across running sessions

Transaction timeouts — per-session tx timeout; expected business exceptions classified apart from real errors

Built on the IBSurgeon driver

firebirdsql-go — IBSurgeon's maintained fork of the popular Go Firebird driver.

Firebird wire protocol
SRP and legacy authentication
Wire encryption and compression

Registered as driver firebirdsql for Go's database/sql.

Metrics & reports

Live counters in the UI and API; durable artifacts on disk after every run.

TPS & success — transactions per second, per session and fleet-wide

Latency p50 / p95 / p99 — transaction latency percentiles

Error classification — expected (business) vs unexpected, with top operations

SQL error log — every failed statement logged with context

Per-run report directory

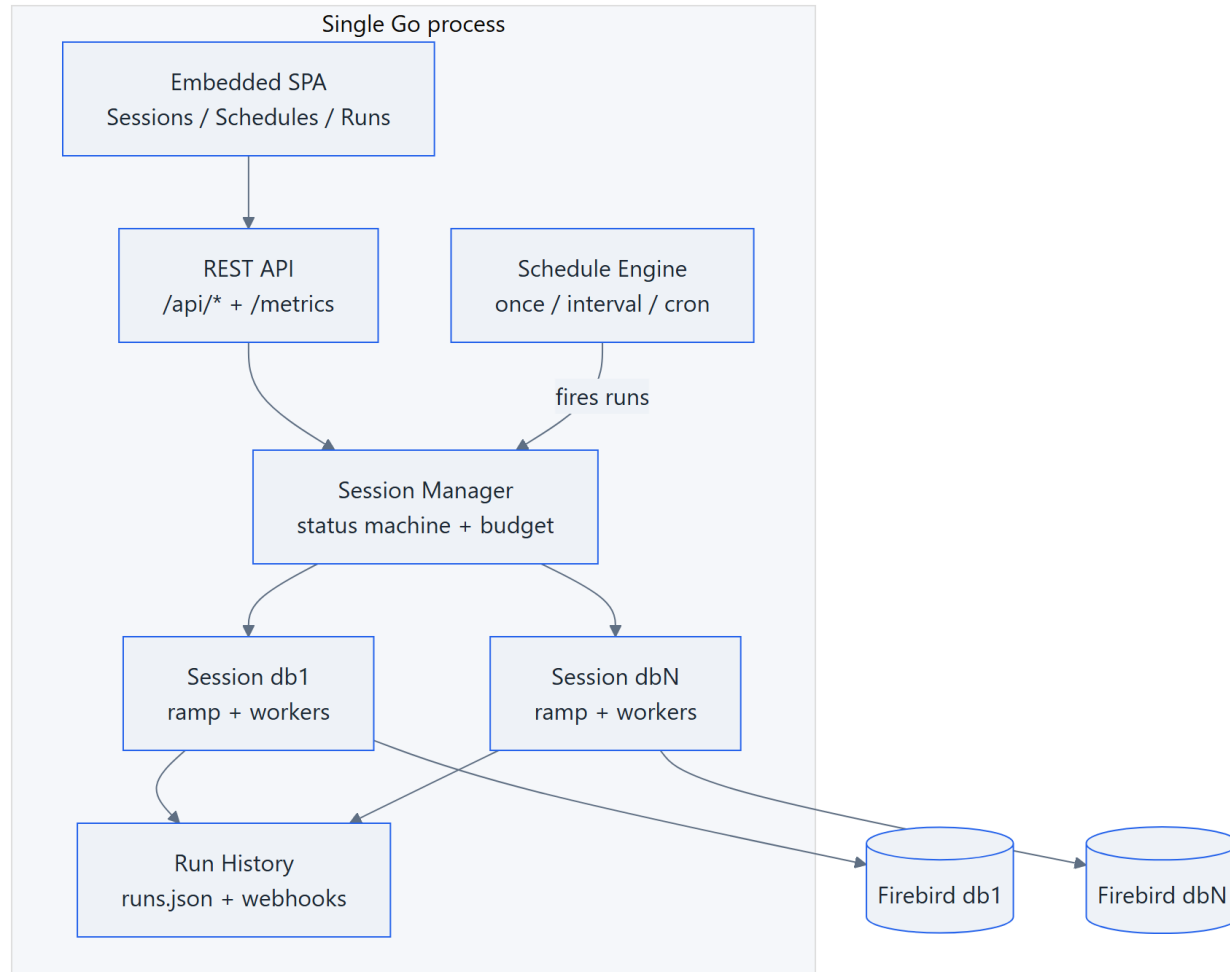
```
reports/db1_EMPLOYEE.FDB/20260829-201952/  
  results_summary.txt  
  results_performance.txt  
  results_operations.txt  
  results_status.txt  
  results.csv + results.csv_latency.txt  
  sql_errors.log
```

```
GET /api/sessions/{id}/report/{file}
```

Download any file over the API — no filesystem access needed.

Control plane architecture

One Go process serves the UI, the REST API, the scheduler and the load itself.



- Embedded SPA — no separate web server
- REST API — same process, same binary
- Schedule engine — persisted, restart-safe
- Session manager — status + connection budget
- Run history — records, webhooks, retention

UI approach — a thin client over the API

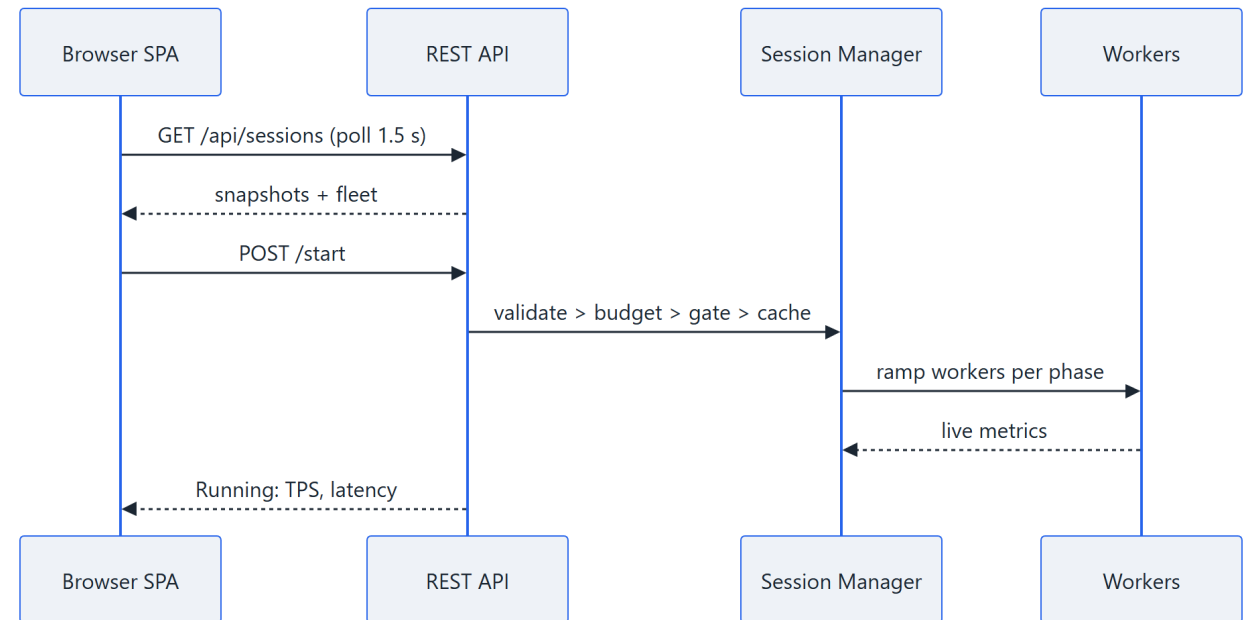
The UI is just another API client: everything it shows comes from the same REST endpoints.

Embedded, zero-install — vanilla-JS SPA inside the exe
— no framework, no build step

Live, not frozen — polls the API every 1.5 s; per-row
start/pause/stop

Schedules & Runs tabs — create schedules, run now,
inspect per-session outcomes

Operator-friendly — details drawer, light/dark themes,
bearer-token prompt



API-first: everything is an endpoint

The API is the product surface; the UI proves it. Full guide with curl and PowerShell recipes in API.md.

Sessions

```
GET /api/sessions
GET /api/sessions/{id}
POST .../start stop pause resume
POST .../validate
PATCH /api/sessions/{id}
start-all / stop-all / pause-all
```

Schedules

```
GET /api/schedules
POST /api/schedules
PATCH / DELETE /{id}
POST /{id}/trigger
GET /{id}/runs
```

Runs & reports

```
GET /api/runs?session=&since=
GET /api/runs/{id}
POST /api/runs/{id}/cancel
GET .../{id}/report/{file}
```

Discovery & ops

```
POST /api/discover
GET/PUT /api/config
GET /api/health /api/version
GET /metrics (Prometheus)
```

--ui-token — bearer auth

--ui-auth-all — protect GETs too

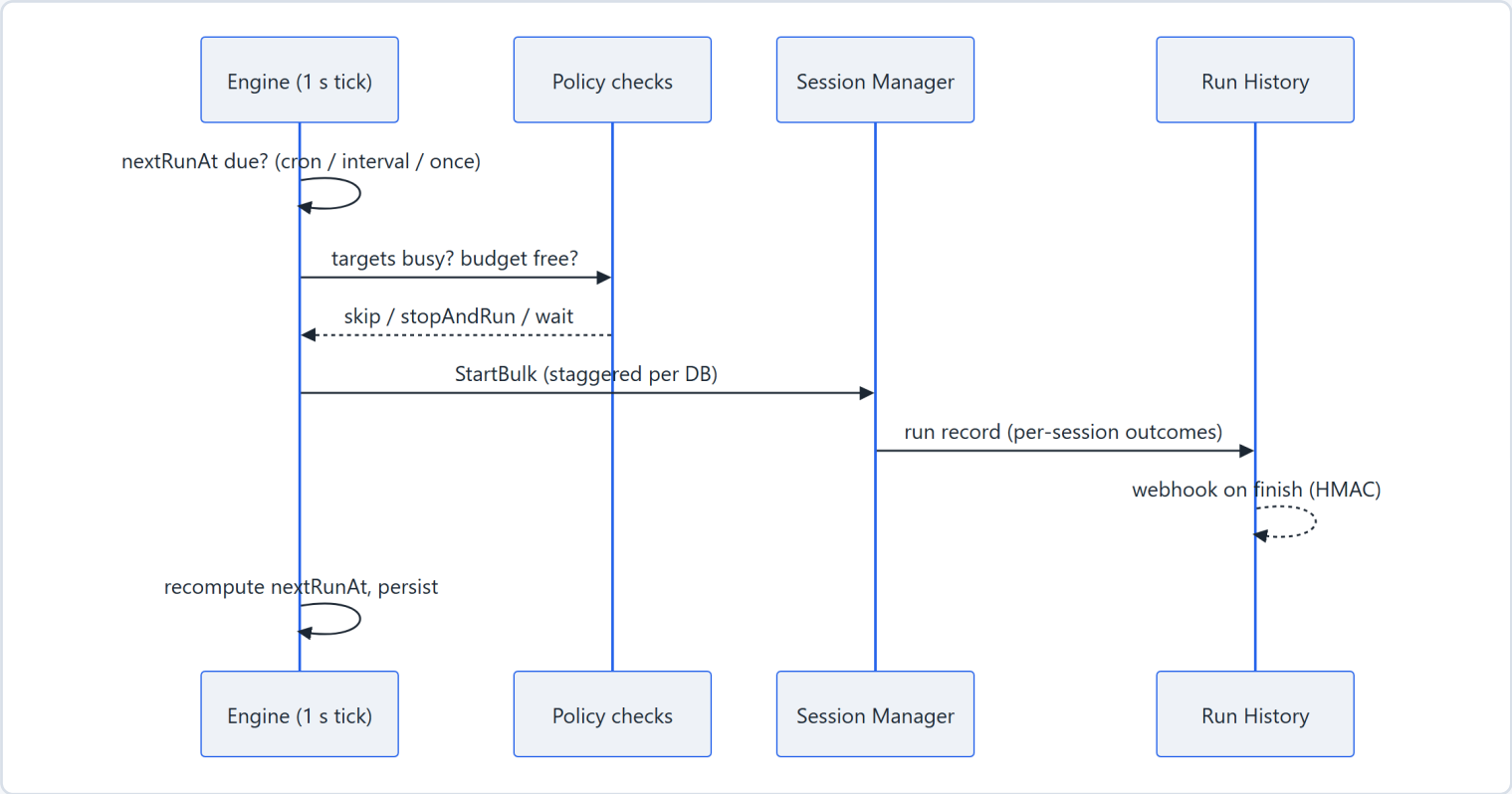
--api-only — headless for CI

wait:false — async start → 202

--cors-origin — browser clients

Scheduled runs — load without a human

Persisted schedules fire runs automatically; every fire is recorded, skippable and budget-aware.



once / interval / cron + timezone

ifRunning: skip or stopAndRun

ifBudgetFull: wait (timeout) or skip

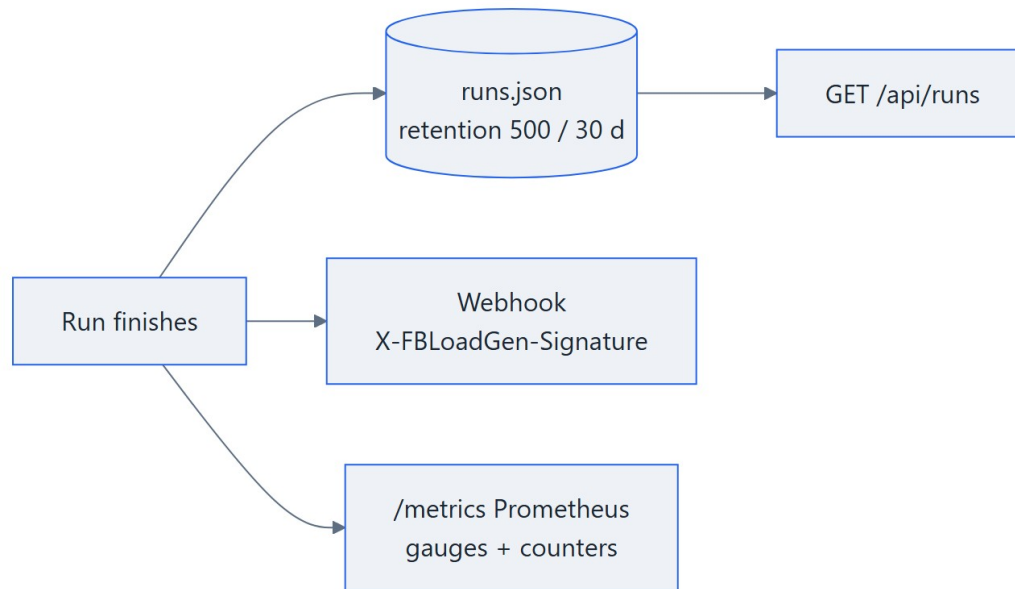
staggerSec between per-DB starts

survives restarts; catchUp for missed fires

optional webhook per schedule

Run history & observability

Every execution — scheduled or manual — becomes a queryable record.



Per-session outcomes — status, TPS, success/errors, report dir, skip reasons

Retention — 500 runs / 30 days, atomic writes

Honest history — process restarted mid-run → run marked cancelled

Webhooks — HMAC-signed JSON on every finished run

Prometheus — fleet, per-session and schedule fire metrics

Query it

```
curl $BASE/api/runs?limit=10
curl $BASE/api/runs/$RUN
curl $BASE/api/schedules

# runs?outcome=ok&since=...
# per-session:
curl $BASE/api/sessions/$ID/runs
```

Outcome: running / ok / partial / failed / cancelled / skipped

AI in the loop — agents run the load spikes

The API is a machine contract: JSON everywhere, an OpenAPI spec, signed webhooks. An agent can close the loop unattended.



Machine-readable contract — the agent reads api/openapi.yaml — endpoints, schemas and status codes without human docs

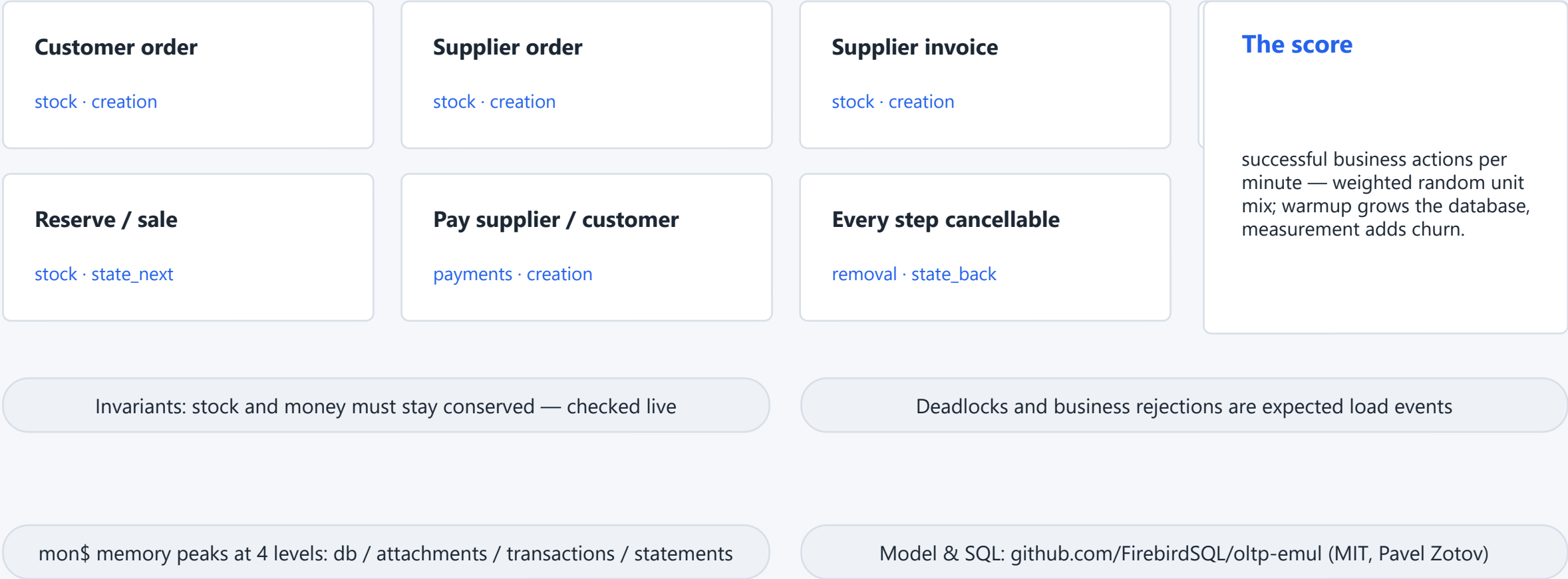
Headless by design — --api-only serves JSON only; async start returns 202; /api/health for liveness

Push, not poll — the HMAC-signed run.finished webhook announces every completed run

Bounded autonomy — server-side policies cap every spike: ifRunning, budget wait, time limits

OLTP-EMUL — a business process, not just SQL

Borrows the FirebirdSQL oltp-emul model (MIT, Pavel Zotov): a car-service supply business runs inside the database as stored procedures.



20 units, 1 transaction each. The database is the application — the generator decides what happens next.

Provision → run → report — one lifecycle

The OLTPÉMUL tab drives the whole flow: provision a benchmark database, run it, read the score.

1 · Provision

```
fb-loadgen provision \  
  --dsn "127.0.0.1/3055:C:\\data\  
\\olt.fdb" \  
  --working-mode SMALL_01 \  
  --init-docs 3000
```

create DB → DDL → procedures → settings → dictionaries → documents. Async job with progress; the database self-registers in the fleet.

2 · Run

```
OLTPÉMUL tab → Start  
# unit mix editable live  
PUT /api/sessions/$ID/emul/weights
```

Schema guard first. Weighted random units, one transaction each, per-unit outcomes: ok / conflict / rejected / failure.

3 · Report

```
results_emul.txt  
+ live score, per-unit table,  
  memory peaks, invariants
```

Score (actions/min) frozen into the report dir and run history — compare runs over time.

works headless too: the same REST API drives provisioning, starting and stopping

The OLTPEMUL dashboard — truth per unit

Live score, memory peaks, invariant status and the per-unit outcome table — one tab, one database, one click to start.

IBSurgeon Firebird Load Generatorlocalhost:3055 - SYSDBA - max 20

CONNECTION

HostlocalhostPort3055UserSYSDBAPasswordblank = keep seMax total conns20Save connectionsaved → fb-loadgen.ui.json

DISCOVERY

Scan rootC:\Database\TEST_10Subdir filteroptional under scan rooMask*.fdbDiscoverValidate allPurge missing

FLEET

Start AllPause AllStop All

SessionsSchedulesRunsOLTPEMUL

Databaseoltpemul_ui2.fdb ✓ oltp-emulRefreshRunning — mainApply settingsStartPauseResumeStop

PROVISION NEW DATABASE

DSN (HOST/PORT)SERVER PATH127.0.0.1/3055;C:\data\oltpemul.fdbUSERSYSDBAPASSWORDWORKING MODESMALL_01INIT DOCS3000PAGE SIZE8192ProvisionCancel job

RUN SETTINGS

PROFILEoltp-emulCONN MIN1CONN MAX (BUDGET)1 (budget)WARMUP S10MAIN S0COOLDOWN S5THINK MS50IN VARIANT EVERY S5MONITOR EVERY S10Apply settings

UNIT MIX (WEIGHTS)

UNITKINDWEIGHTSave weights

LIVE RESULTS

330ops/min1643/1655 units OK

PER-UNIT BREAKDOWN

UNIT	KIND	OK	CONFLICT	REJ.	FAIL	AVG MS	MAX MS
srv_recalc_idx_stat	service	197	0	0	0	105	151
sp_client_order	creation	169	0	0	0	114	139
sp_reserve_write_off	state_next	167	0	0	0	126	254
sp_pay_from_customer	creation	165	0	0	0	114	143
sp_add_invoice_to_stock	state_next	138	0	0	0	215	471
sp_supplier_invoice	creation	137	0	0	0	124	151

score sparkline per 10 s interval

memory peaks: db / att / trn / stmt

per-unit ok · conflict · rejected · fail
+ avg/max ms

last runs: score comparison over
time

Typical scenarios

Nightly soak

```
curl -X POST $BASE/api/schedules -d '{
  "trigger": {"type":"cron",
    "cron":"0 2 * * *"},
  "run": {"timeLimitMin":30},
  "notifyUrl":"https://hooks/..."}'
```

Every night at 02:00, 30 minutes, webhook tells you the outcome

CI spike test

```
fb-loadgen --ui --api-only &
curl -X POST $BASE/api/schedules/
  $SID/trigger      # fire now
curl $BASE/api/runs/$RUN  # outcome
# fail the build on outcome != ok
```

Headless process in CI; build fails if the run fails

Fleet health

```
curl $BASE/api/fleet
curl $BASE/metrics | grep fbloadgen

# running, budget, TPS, errors,
# next scheduled fire – in Prometheus
```

One scrape shows the whole fleet's load state

Same API drives all three — the UI, CI and cron are interchangeable clients.

Where we are, where we go

Shipped

- Scheduled runs (once / interval / cron)
- Run history with per-session outcomes
- Webhooks + Prometheus /metrics
- API hardening: auth, CORS, headless mode
- OpenAPI 3.1 spec for the whole API
- oltp-emul business-process mode + OLTPÉMUL dashboard
- CI (vet + race + 3 OS) and multi-platform releases
- IBSurgeon driver fork under the hood

Next

- Secure credential storage (no plaintext settings)
- Schedule chains (run A, then B)
- Your use case here — tell us what you need

Docs: [README.md](#) · [API.md](#) · [IMPROVEMENTS.md](#) **Start:** `fb-loadgen --ui`

Appendix — quick reference

```
# run
fb-loadgen --profile write-heavy --dsn "host/3050:path\db.fdb"
fb-loadgen --ui [--api-only] [--ui-token T] [--ui-addr 127.0.0.1:9000]

# control
GET /api/sessions          GET /api/fleet
POST /api/sessions/{id}/start {"timeLimitMin": 15, "wait": false}
POST /api/sessions/{id}/stop | pause | resume | validate
POST /api/sessions/start-all | stop-all | pause-all

# schedule + history
GET|POST /api/schedules      PATCH|DELETE /api/schedules/{id}
POST /api/schedules/{id}/trigger
GET /api/runs | /api/runs/{id} | POST /api/runs/{id}/cancel
GET /api/sessions/{id}/runs | /report | /report/{file}
GET /api/health | /api/version | /metrics
```